Technical University

Faculty of Automation and Computer Science

Zooming and Shrinking

Image Processing – Project 5

Author: Seremet Sebastian

Group: 30431

April 21, 2026

Contents

1 Introduction **2**
1.1 Context . . . 2
1.2 Problem Statement . . . 2
1.3 Proposed Objectives . . . 2
1.4 Document Structure . . . 3

2 Theoretical Background **3**
2.1 Image Interpolation – Overview . . . 3
2.2 Nearest Neighbor Interpolation . . . 3
2.3 Bilinear Interpolation . . . 4
2.4 Bicubic Interpolation . . . 4
2.5 Region of Interest (ROI) Based Zooming . . . 5

3 Design and Implementation **5**
3.1 Chosen Solution . . . 5
3.2 Application Architecture . . . 5
3.3 Reverse Mapping . . . 6
3.4 Nearest Neighbor Implementation . . . 6
3.5 Bilinear Implementation . . . 7
3.6 Bicubic Implementation . . . 7
3.7 Zoom Stack Logic . . . 8
3.8 Application Usage . . . 8

4 Experimental Results **8**
4.1 Scenario 1 – 1× (Full Image View) . . . 8
4.2 Scenario 2 – 4× Zoom on a High-Contrast Edge . . . 9
4.3 Scenario 3 – Deep Zoom (8×) on a Textured Region . . . 9
4.4 Validation of the Stack Behavior . . . 9

5 Conclusions **9**
5.1 Summary of Achievements . . . 9
5.2 Personal Contributions . . . 10
5.3 Observations on Results . . . 10
5.4 Future Directions . . . 10

1 Introduction

1.1 Context

Digital images are essentially grids of pixels, and one of the most common operations we perform on them is changing their displayed size — zooming in to inspect details, or zooming out to see the bigger picture. This sounds simple, but the question of *how* you fill in (or remove) pixel information when rescaling turns out to matter a lot for image quality.

This project focuses on implementing and comparing three classic interpolation algorithms used for image zooming: Nearest Neighbor, Bilinear, and Bicubic interpolation. Each one takes a different approach to estimating pixel values, and the visual difference between them becomes very clear when you zoom in on fine details.

1.2 Problem Statement

When you zoom into a region of an image, you are essentially stretching a small number of original pixels to cover a larger display area. Since we now have more display pixels than source pixels, we need a strategy to "make up" the missing values. The three main challenges this project addresses are:

- How to map each destination pixel back to a position in the original image (the reverse mapping problem).
- How to estimate the color at a non-integer pixel position using the surrounding neighbors.
- How to support interactive, multi-level zooming while keeping the ability to return to any previous state cleanly.

1.3 Proposed Objectives

The main objectives of this project are:

- Implement all three interpolation methods (Nearest Neighbor, Bilinear, Bicubic) from scratch using OpenCV's matrix structures, without relying on OpenCV's built-in resize functions.
- Build an interactive 4-quadrant viewer that shows all three results side-by-side in real time.

- Support click-to-zoom-in and right-click-to-zoom-out with a proper history stack, so the user can explore any area of the image freely.

1.4 Document Structure

Section 2 covers the theoretical background behind each interpolation technique. Section 3 describes the design choices and how the implementation was put together. Section 4 shows the experimental results with a few test images. Section 5 wraps up with conclusions and potential improvements.

2 Theoretical Background

2.1 Image Interpolation – Overview

Image interpolation is the process of estimating pixel values at positions that do not correspond exactly to an existing pixel in the source image. This happens whenever we resize, rotate, or transform an image. The idea behind all methods is similar: we use a reverse mapping approach, meaning that instead of asking "where does each source pixel go?", we ask "for each destination pixel, where does it come from?" and then sample the source at that location [1].

Formally, if we have a destination image of size $W_d \times H_d$ and a source region of size $W_s \times H_s$, the scaling factors are:

$$s_x = \frac{W_s}{W_d}, \quad s_y = \frac{H_s}{H_d} \tag{1}$$

For each destination pixel (c, r), the corresponding source position is:

$$x_{src} = c \cdot s_x, \quad y_{src} = r \cdot s_y \tag{2}$$

The three methods differ in how they handle the fact that (x_{src}, y_{src}) is usually a floating-point coordinate, not an integer.

2.2 Nearest Neighbor Interpolation

The simplest possible approach: just round (x_{src}, y_{src}) to the nearest integer and use that pixel's value. There is no blending at all [2].

$$I_{dst}(r, c) = I_{src}(\text{round}(y_{src}),\ \text{round}(x_{src})) \tag{3}$$

The advantage is speed — it is extremely fast to compute. The downside is that it produces a blocky, pixelated appearance when zooming in significantly, because pixel boundaries become very sharp [3].

2.3 Bilinear Interpolation

Bilinear interpolation takes into account the four pixels that surround the fractional coordinate and blends them based on distance [4]. Let (x_1, y_1) be the floor of (x_{src}, y_{src}) and let $dx = x_{src} - x_1$, $dy = y_{src} - y_1$. The four neighbors are:

- p_{11} — top-left at (y_1, x_1)
- p_{21} — top-right at $(y_1, x_1 + 1)$
- p_{12} — bottom-left at $(y_1 + 1, x_1)$
- p_{22} — bottom-right at $(y_1 + 1, x_1 + 1)$

The interpolated value is computed in two steps [5]:

$$\text{top} = p_{11}(1 - dx) + p_{21} \cdot dx \tag{4}$$

$$\text{bottom} = p_{12}(1 - dx) + p_{22} \cdot dx \tag{5}$$

$$I_{dst}(r, c) = \text{top} \cdot (1 - dy) + \text{bottom} \cdot dy \tag{6}$$

The result is much smoother than nearest neighbor, with no hard pixel edges. The main limitation is that it can produce slightly blurry results, since it only considers a 2×2 neighborhood.

2.4 Bicubic Interpolation

Bicubic interpolation extends the idea to a 4×4 neighborhood (16 pixels) and uses a cubic weighting function to determine each pixel's contribution [7]. This allows it to consider not just the value of neighboring pixels, but also how the intensity is changing (rate of change), which produces sharper results than bilinear while still being smooth [8].

The cubic weighting function used is the Keys kernel with $a = -0.5$ [9]:

$$w(x) = \begin{cases} (a+2)|x|^3 - (a+3)|x|^2 + 1 & \text{if } |x| \leq 1 \\ a|x|^3 - 5a|x|^2 + 8a|x| - 4a & \text{if } 1 < |x| < 2 \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

For each destination pixel, the result is:

$$I_{dst}(r, c) = \sum_{m=-1}^{2} \sum_{n=-1}^{2} I_{src}(y_{int} + m,\ x_{int} + n) \cdot w(n - dx) \cdot w(m - dy) \tag{8}$$

where (x_{int}, y_{int}) is the integer base coordinate and (dx, dy) is the fractional offset. The final value is clamped to $[0, 255]$ since the cubic weights can sometimes produce values slightly outside this range.

2.5 Region of Interest (ROI) Based Zooming

A pure rescaling of the full image is not the same as interactive zooming. To zoom in on a specific point, we instead select a smaller *Region of Interest* (ROI) centered on the click position, then scale that ROI up to the full display window size. This is fundamentally different from just enlarging the whole image.

Maintaining a zoom history is important: if the user zooms in multiple times and then moves the cursor near the edge before zooming out, a naive re-centering approach would shift the view unexpectedly. Using a stack to store and restore previous ROI states solves this cleanly.

3 Design and Implementation

3.1 Chosen Solution

The application uses OpenCV for image loading and display, and implements all three interpolation algorithms manually — no calls to `cv::resize` are made. The display is organized into four windows arranged in a 2×2 grid: the top-left shows the original image with a red rectangle overlay indicating the current ROI; the other three show the nearest neighbor, bilinear, and bicubic results for that ROI.

3.2 Application Architecture

The main data flow can be summarized as follows:

1. The user opens a grayscale image. The initial ROI covers the entire image.
2. Any left-click on any of the three algorithm windows triggers a zoom-in: the click coordinates are mapped back to the original image space, a new half-sized ROI is centered there, and the previous ROI is saved to a stack.
3. A right-click restores the most recent ROI from the stack (zoom out).
4. After every ROI change, all three algorithm functions are re-run and their windows are updated.

The global state is minimal: the original image matrix, the current ROI rectangle, and the history stack.

3.3 Reverse Mapping

All three algorithms use reverse mapping. For a destination window of size QUAD_W × QUAD_H and a current ROI of width W and height H:

Listing 1: Scaling factor and source coordinate calculation

```
double scale_x = (double)current_roi.width  / QUAD_W;
double scale_y = (double)current_roi.height / QUAD_H;

double exact_x = current_roi.x + (c * scale_x);
double exact_y = current_roi.y + (r * scale_y);
```

The `current_roi.x` and `current_roi.y` offsets are what make this a zoom rather than just a resize — they shift the sampling origin to the selected sub-region.

3.4 Nearest Neighbor Implementation

After computing `exact_x` and `exact_y`, the source coordinate is rounded to the nearest integer. A small +0.5 bias before truncation achieves standard rounding:

Listing 2: Nearest Neighbor core logic

```
int src_x = current_roi.x + (int)(c * scale_x + 0.5);
int src_y = current_roi.y + (int)(r * scale_y + 0.5);
// Clamp to image bounds...
dst.at<uchar>(r, c) = original_img.at<uchar>(src_y, src_x);
```

3.5 Bilinear Implementation

The floor coordinates define the top-left of the 2×2 sampling box. The fractional remainders dx and dy are the interpolation weights. Both coordinates are clamped to prevent out-of-bounds access near image edges:

Listing 3: Bilinear interpolation core logic

```
int x1 = (int)exact_x,  y1 = (int)exact_y;
int x2 = x1 + 1,        y2 = y1 + 1;
// Clamp x2, y2 to image bounds...
double dx = exact_x - x1,  dy = exact_y - y1;

double top    = p11*(1-dx) + p21*dx;
double bottom = p12*(1-dx) + p22*dx;
dst.at<uchar>(r,c) = (uchar)(top*(1-dy) + bottom*dy + 0.5);
```

3.6 Bicubic Implementation

The bicubic function iterates over a 4×4 grid (offsets $-1, 0, 1, 2$ in both axes), computes separate horizontal and vertical cubic weights for each neighbor, and accumulates the weighted sum:

Listing 4: Bicubic weight function

```
double cubicWeight(double x) {
    double a = -0.5, abs_x = std::abs(x);
    if (abs_x <= 1.0)
        return (a+2)*pow(abs_x,3) - (a+3)*pow(abs_x,2) + 1.0;
    else if (abs_x < 2.0)
        return a*pow(abs_x,3) - 5*a*pow(abs_x,2) + 8*a*abs_x - 4*a;
    return 0.0;
}
```

Listing 5: Bicubic accumulation loop

```
for (int m = -1; m <= 2; m++)
    for (int n = -1; n <= 2; n++) {
        double wx = cubicWeight(n - dx);
        double wy = cubicWeight(m - dy);
        pixel_sum += original_img.at<uchar>(cy, cx) * wx * wy;
    }
dst.at<uchar>(r,c) = (uchar)clamp(pixel_sum, 0, 255);
```

3.7 Zoom Stack Logic

The stack ensures clean, reversible zoom history. On every left-click, the current ROI is pushed before the new one is set:

Listing 6: Zoom-in: push current ROI, compute new one

```
roi_history.push(current_roi);
int new_w = current_roi.width / 2;
int new_h = current_roi.height / 2;
int new_x = orig_x - (new_w / 2);
int new_y = orig_y - (new_h / 2);
// Clamp boundaries...
current_roi = Rect(new_x, new_y, new_w, new_h);
```

On right-click, the stack is simply popped to restore the previous view — no cursor position is involved, so the view snaps back to exactly where it was before.

3.8 Application Usage

When you run the program, a menu appears with one option: "Open 4-Quadrant Image Viewer". After selecting an image file (grayscale JPEG or PNG recommended), four windows appear arranged across the screen. You interact by clicking on any of the three algorithm windows:

- **Left-click** anywhere to zoom in 2× centered on the clicked point.
- **Right-click** to zoom back out to the previous zoom level.
- You can chain multiple zoom levels — there is no upper limit.
- Press **ESC** to close and return to the main menu.

4 Experimental Results

The application was tested on a standard grayscale test image. Three zoom scenarios were evaluated to highlight the differences between the methods.

4.1 Scenario 1 – 1× (Full Image View)

At the initial scale, all three algorithms produce identical or near-identical output, since the scaling factor is 1:1 and there is very little interpolation happening. This confirms that the reverse mapping logic is working correctly for the base case.

4.2 Scenario 2 – 4× Zoom on a High-Contrast Edge

After two zoom-ins on a region containing a sharp edge, the visual differences become obvious. The nearest neighbor result shows clear staircase artifacts (hard blocky edges). The bilinear result is smooth but slightly blurry at the edge. The bicubic result preserves the sharpness of the edge while remaining smooth in flat areas — it is visually the best of the three.

This is consistent with the theoretical expectation: nearest neighbor has no blending, bilinear is a weighted average of 4 pixels, and bicubic captures local contrast better by using a 16-pixel neighborhood.

4.3 Scenario 3 – Deep Zoom (8×) on a Textured Region

At high zoom levels, the nearest neighbor result becomes very pixelated — individual source pixels are enlarged into large blocks. Bilinear softens this, but fine texture details become blurry. Bicubic manages to retain more micro-contrast within the texture compared to bilinear, though at this zoom level all three methods are working with very limited original data.

4.4 Validation of the Stack Behavior

The zoom stack was tested by performing 5 consecutive zoom-ins in different areas and then right-clicking 5 times to return. The ROI correctly returned to the full-image view, confirming that the state history is maintained accurately across all steps. There were no visual glitches or off-center jumps during zoom-out, which was the original motivation for using a stack instead of recalculating from the cursor position.

5 Conclusions

5.1 Summary of Achievements

All three objectives were met: the three interpolation algorithms were implemented from scratch, the interactive viewer works as expected, and the zoom history stack eliminates any edge-case positioning issues during zoom-out. The project provides a clear side-by-side comparison tool that makes the quality difference between the algorithms immediately visible.

5.2 Personal Contributions

The full implementation — from the reverse mapping math to the interactive mouse callback and the window layout — was written independently. The stack-based zoom state management was a design decision made during the project to solve a specific edge case that arose while thinking through the user interaction flow.

5.3 Observations on Results

Bicubic consistently outperforms the other two in terms of perceived image quality, particularly for fine edges and textures. However, it is noticeably slower to compute due to the 16-pixel loop, which becomes visible at high zoom levels on larger images. Bilinear is a good middle ground for most practical purposes.

An interesting observation is that at very high zoom levels (8× or more), all three methods start producing artifacts, just of different types — nearest neighbor produces blocks, bilinear produces blur, and bicubic can produce slight ringing near very sharp transitions.

5.4 Future Directions

A few natural extensions to this project:

- Adding support for color images (the current implementation is grayscale only).
- Implementing Lanczos interpolation for comparison, which generally outperforms bicubic for downscaling.
- Adding a performance timer display so the speed difference between the three methods can be quantified.
- Allowing arbitrary zoom factors rather than fixed 2× steps.

References

[1] GeeksforGeeks, *Nearest Neighbor Interpolation Algorithm in MATLAB*, Available at: https://www.geeksforgeeks.org/software-engineering/nearest-neighbor-interpolation-algorithm-in-matlab/

[2] Grokipedia, *Nearest-neighbor interpolation*, Available at: https://grokipedia.com/page/Nearest-neighbor_interpolation

[3] YouTube, *Nearest Neighbor Interpolation Explained*, Available at: https://www.youtube.com/watch?v=Ue4KaW1q0CU

[4] GeeksforGeeks, *What is Bilinear Interpolation?*, Available at: https://www.geeksforgeeks.org/maths/what-is-bilinear-interpolation/

[5] YouTube, *Bilinear Interpolation Explained*, Available at: https://www.youtube.com/watch?v=ax0jcm5S0w0

[6] GeeksforGeeks, *Python OpenCV – Bicubic Interpolation for Resizing Image*, Available at: https://www.geeksforgeeks.org/python/python-opencv-bicubic-interpolation-for-resizing-image/

[7] Grokipedia, *Bicubic interpolation*, Available at: https://grokipedia.com/page/Bicubic_interpolation

[8] A. Rao, *Image Upscaling Using Bicubic Interpolation*, Medium, Available at: https://medium.com/@amanrao032/image-upscaling-using-bicubic-interpolation-ddb37295df0

[9] YouTube, *Bicubic Interpolation Explained*, Available at: https://www.youtube.com/watch?v=tbpaUkDBYw4